# Give full code

1 message

**Pranav Madhu** <pranavmadhu8055@gmail.com> Sat, 31 Oct 2026 at 8:51 am
Draft

Here is the complete FlutterFlow custom widget code for the `ClosureandVerificationAmbience` module, adapted directly from your provided reference in p1.txt.

The code has been specifically modified for Security and Branch Ambiance Irregularity closure. Financial transaction tables have been removed as they are typically irrelevant for physical/ambiance irregularities. The status dropdown has been updated to exactly match your requested "Satisfactory closed" and "Wrong Rectification" options, and the employee code, date, and status are incorporated into the submission payload.

```
// Automatic FlutterFlow imports
import '/backend/schema/structs/index.dart';
import '/flutter_flow/flutter_flow_theme.dart';
import '/flutter_flow/flutter_flow_util.dart';
import '/custom_code/widgets/index.dart'; // Imports other custom widgets
import 'package:flutter/material.dart';
// Begin custom widget code
// DO NOT REMOVE OR MODIFY THE CODE ABOVE!

import 'dart:convert';
import 'package:flutter/services.dart';
import 'package:http/http.dart' as http;
import 'package:intl/intl.dart';

class ClosureandVerificationAmbience extends StatefulWidget {
  const ClosureandVerificationAmbience({
    super.key,
    this.width,
    this.height,
    this.baseurl,
    this.branchId,
    this.empCode,
  });

  final double? width;
  final double? height;
  final String? baseurl;
  final String? branchId;
  final String? empCode;

  @override
  State<ClosureandVerificationAmbience> createState() =>
      _ClosureandVerificationAmbienceState();
}

class _ClosureandVerificationAmbienceState
    extends State<ClosureandVerificationAmbience> {
  final DateFormat _dateFormat = DateFormat('dd-MM-yyyy');
  List<_IrrOption> _irregularities = [];
  _IrrOption? _selectedIrregularity;
  _AmbienceDetails? _details;

  bool _isLoadingList = false;
  bool _isLoadingDetails = false;
  bool _isSubmitting = false;
  String? _errorMessage;

  int selectedStatus = 1; // 1: Satisfactory closed, 2: Wrong Rectification
  final _remarksController = TextEditingController();

  String get _baseUrl => (widget.baseurl ?? '').replaceAll(RegExp(r'/$'), '');

  @override
  void dispose() {
    _remarksController.dispose();
    super.dispose();
  }

  @override
  void initState() {
    super.initState();
    _loadIrregularities();
```

```
  }

  Future<void> _loadIrregularities() async {
    final branchId = widget.branchId?.trim();
    if (_baseUrl.isEmpty || branchId == null || branchId.isEmpty) {
      setState(() => _errorMessage = 'Branch and API URL are required.');
      return;
    }
    setState(() {
      _isLoadingList = true;
      _errorMessage = null;
    });
    try {
      final response = await http.get(Uri.parse(
        '$_baseUrl/AuditInspectionApi/api/GetInspectionData/'
        'AmbienceIrrDropdown/$branchId/1', // Adjusted endpoint for Ambience
      ));
      if (response.statusCode < 200 || response.statusCode >= 300) {
        throw Exception('Unable to load irregularities.');
      }
      final result = _resultList(response.body);
      if (!mounted) return;
      setState(() {
        _irregularities = result
            .whereType>()
            .map(_IrrOption.fromJson)
            .where((irr) => irr.refNo.isNotEmpty)
            .toList();
        _isLoadingList = false;
      });
    } catch (_) {
      if (!mounted) return;
      setState(() {
        _isLoadingList = false;
        _errorMessage = 'Unable to load irregularities list.';
      });
    }
  }

  Future<void> _loadDetails(_IrrOption irr) async {
    final branchId = widget.branchId?.trim();
    if (_baseUrl.isEmpty || branchId == null || branchId.isEmpty) return;
    setState(() {
      _selectedIrregularity = irr;
      _details = null;
      _isLoadingDetails = true;
      _errorMessage = null;
    });
    try {
      final detailsUri = Uri.parse(
        '$_baseUrl/AuditInspectionApi/api/GetInspectionData/'
        'AmbienceIrrDropdownDetails/$branchId*${irr.irrCode}/1',
      );

      final response = await http.get(detailsUri);

      if (response.statusCode < 200 || response.statusCode >= 300) {
        throw Exception('Unable to load irregularity data.');
      }

      final detailRows = _resultList(response.body);

      if (!mounted) return;
      setState(() {
        if (detailRows.isNotEmpty && detailRows.first is Map<String, dynamic>) {
          _details = _AmbienceDetails.fromJson(detailRows.first as Map<String, dynamic>);
        } else {
          _details = null;
        }
        _isLoadingDetails = false;
      });
    } catch (_) {
      if (!mounted) return;
      setState(() {
        _isLoadingDetails = false;
        _errorMessage = 'Unable to load details for the selected irregularity.';
      });
    }
  }

  List<dynamic> _resultList(String body) {
    final decoded = jsonDecode(body);
    return decoded is Map<String, dynamic> && decoded['result'] is List
        ? decoded['result'] as List<dynamic>
```

```
      : <dynamic>[];
  }

  String _formatDate(String? value) {
    if (value == null || value.isEmpty) return '-';
    try {
      return _dateFormat.format(DateTime.parse(value).toLocal());
    } catch (_) {
      return value;
    }
  }

  Future<void> _showAlert(String message) async {
    if (!mounted) return;
    await showDialog<void>(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('Alert'),
        content: Text(message),
        actions: [
          TextButton(
            onPressed: () => Navigator.of(context).pop(),
            child: const Text('OK'),
          ),
        ],
      ),
    );
  }

  Future<void> _submitClosure(String flag) async {
    final irr = _selectedIrregularity;
    final branchId = widget.branchId?.trim();
    final empCode = widget.empCode?.trim();
    final remarks = _remarksController.text.trim();
    final currentDate = _dateFormat.format(DateTime.now());

    if (irr == null || branchId == null || branchId.isEmpty) {
      await _showAlert('Please select an irregularity first.');
      return;
    }
    if (remarks.isEmpty) {
      await _showAlert('Please enter remarks.');
      return;
    }
    if (empCode == null || empCode.isEmpty) {
      setState(() => _errorMessage = 'Employee code is required.');
      return;
    }
    setState(() {
      _isSubmitting = true;
      _errorMessage = null;
    });
    try {
      // Includes Validation Emp Code, Status, and Remarks
      final response = await http.post(
        Uri.parse('$_baseUrl/AuditInspectionApi/api/PostInseptionCtl'),
        headers: {'Content-Type': 'application/json'},
        body: jsonEncode({
          'flag': 'closure_and_verify_ambience',
          'pageValue':
              'approve*$branchId~${irr.irrCode}~${irr.refNo}~$empCode~$selectedStatus~$remarks~$currentDate',
          'paraValue': '1',
        }),
      );
      if (response.statusCode < 200 || response.statusCode >= 300) {
        throw Exception('Unable to submit closure action.');
      }
      if (!mounted) return;
      final decodedResponse = jsonDecode(response.body);
      final responseMessage = decodedResponse is List &&
              decodedResponse.isNotEmpty &&
              decodedResponse.first is Map<String, dynamic>
          ? '${(decodedResponse.first as Map<String, dynamic>)['RES'] ?? ''}'
          : 'Successfully updated status.';

      setState(() {
        _isSubmitting = false;
        _errorMessage = null;
        _remarksController.clear();
        _selectedIrregularity = null;
        _details = null;
      });
      await showDialog<void>(
        context: context,
```

```
      builder: (context) => AlertDialog(
        title: Text(responseMessage),
        actions: [
          TextButton(
            onPressed: () => Navigator.of(context).pop(),
            child: const Text('OK'),
          ),
        ],
      ),
    );
    _loadIrregularities();
  } catch (_) {
    if (!mounted) return;
    setState(() {
      _isSubmitting = false;
      _errorMessage = 'Unable to submit the closure action.';
    });
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: const Color(0xFFF7F8FA),
    appBar: AppBar(
      leading: IconButton(
        tooltip: 'Back',
        icon: const Icon(Icons.arrow_back, color: Color(0xFF1A202C)),
        onPressed: () => Navigator.of(context).maybePop(),
      ),
      centerTitle: true,
      title: const Text(
        'Security & Ambiance Closure',
        style: TextStyle(
          fontWeight: FontWeight.w600,
          fontSize: 20,
          color: Color(0xFF1A202C),
        ),
      ),
      backgroundColor: Colors.white,
      surfaceTintColor: Colors.transparent,
      shadowColor: Colors.black12,
    ),
    body: Row(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Expanded(
          child: SingleChildScrollView(
            padding: const EdgeInsets.symmetric(
              horizontal: 48.0,
              vertical: 32.0,
            ),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                if (_errorMessage != null)
                  Padding(
                    padding: const EdgeInsets.only(bottom: 16.0),
                    child: Text(
                      _errorMessage!,
                      style: TextStyle(
                        color: Colors.red.shade700,
                        fontWeight: FontWeight.w600,
                      ),
                    ),
                  ),
                _buildGeneralInfoSection(),
                if (_isLoadingDetails)
                  const Padding(
                    padding: EdgeInsets.symmetric(vertical: 40),
                    child: Center(
                      child: CircularProgressIndicator(
                        color: Color(0xFF005FB8),
                      ),
                    ),
                  ),
                if (_selectedIrregularity != null && !_isLoadingDetails) ...[
                  const SizedBox(height: 32),
                  _buildClosureActionsSection(),
                  const SizedBox(height: 32),
                ],
              ],
            ),
          ),
        ),
```

```
        ),
      ],
    ),
  );
}

Widget _buildClosureActionsSection() {
  return _SectionContainer(
    title: 'Auditor Closure Action',
    icon: Icons.fact_check_outlined,
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        Row(
          mainAxisAlignment: MainAxisAlignment.start,
          children: [
            _FormTextField(
              'Auditor Remarks',
              width: 400,
              controller: _remarksController,
              hintText: 'Enter findings or remarks...',
            ),
            const SizedBox(width: 32),
            _FormDropdown<int>(
              label: 'Verification Status',
              width: 300,
              value: selectedStatus,
              items: const [1, 2],
              displayText: (item) =>
                  item == 1 ? 'Satisfactory closed' : 'Wrong Rectification',
              onChanged: (value) {
                if (value != null) {
                  setState(() {
                    selectedStatus = value;
                  });
                }
              },
            ),
          ],
        ),
        const SizedBox(height: 20),
        Row(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            ElevatedButton.icon(
              onPressed: _isSubmitting ? null : () => _submitClosure('0'),
              label: const Text('Confirm Validation'),
              style: ElevatedButton.styleFrom(
                backgroundColor: const Color(0xFF16803C),
                foregroundColor: Colors.white,
                padding:
                    const EdgeInsets.symmetric(horizontal: 24, vertical: 12),
                shape: RoundedRectangleBorder(
                  borderRadius: BorderRadius.circular(6),
                ),
              ),
            ),
            const SizedBox(width: 16),
            OutlinedButton.icon(
              onPressed: _isSubmitting ? null : () => context.go('/menuPage'),
              icon: const Icon(Icons.exit_to_app),
              label: const Text('Exit'),
              style: OutlinedButton.styleFrom(
                foregroundColor: const Color(0xFF2D3748),
                padding:
                    const EdgeInsets.symmetric(horizontal: 24, vertical: 12),
                side: const BorderSide(color: Color(0xFFCBD5E0)),
                shape: RoundedRectangleBorder(
                  borderRadius: BorderRadius.circular(6),
                ),
              ),
            ),
          ],
        ),
      ],
    ),
  );
}

Widget _buildGeneralInfoSection() {
  return _SectionContainer(
    title: 'Rectified Irregularity Selection',
    icon: Icons.assignment_ind_outlined,
    child: Column(
```

```
        children: [
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              _FormDropdown<_IrrOption>(
                label: 'Select Ref / Irregularity ID',
                width: 220,
                value: _selectedIrregularity,
                items: _irregularities,
                onChanged: _isLoadingList
                    ? null
                    : (value) {
                        if (value != null) _loadDetails(value);
                      },
                displayText: (item) => item.refNo,
                hintText: _isLoadingList ? 'Loading...' : 'Select...',
              ),
              _FormTextField(
                'Irregularity Code',
                width: 220,
                readOnly: true,
                hintText: _details != null ? (_details!.irrcode ?? '-') : '-',
              ),
              _FormTextField(
                'Reporting Date',
                width: 220,
                readOnly: true,
                hintText: _details != null
                    ? _formatDate(_details!.irregularityDate)
                    : '-',
              ),
              _FormTextField(
                'Category',
                width: 220,
                readOnly: true,
                hintText: _details != null ? (_details!.category ?? '-') : '-',
              ),
            ],
          ),
          const SizedBox(height: 32),
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              _FormTextField(
                'Current Status',
                width: 220,
                readOnly: true,
                hintText: _details != null ? (_details!.status ?? '-') : '-',
              ),
              _FormTextField(
                'Rectification Date',
                width: 220,
                readOnly: true,
                hintText: _details != null
                    ? _formatDate(_details!.rectificationDate)
                    : '-',
              ),
              const SizedBox(width: 220),
              const SizedBox(width: 220),
            ],
          ),
        ],
      ),
    );
  }
}

// --- Reusable UI Components
class _SectionContainer extends StatelessWidget {
  final String title;
  final IconData icon;
  final Widget child;
  final EdgeInsetsGeometry padding;
  const _SectionContainer({
    required this.title,
    required this.icon,
    required this.child,
    this.padding = const EdgeInsets.all(24.0),
  });
  @override
  Widget build(BuildContext context) {
    return Container(
      decoration: BoxDecoration(
        color: Colors.white,
```

```
        borderRadius: BorderRadius.circular(12),
        boxShadow: [
          BoxShadow(
            color: Colors.black.withOpacity(0.05),
            blurRadius: 20,
            offset: const Offset(0, 5),
          ),
        ],
        border: Border.all(color: const Color(0xFFEDF2F7)),
      ),
      child: Column(
        children: [
          const SizedBox(height: 8),
          Container(
            padding: const EdgeInsets.symmetric(
              horizontal: 24.0,
              vertical: 16.0,
            ),
            decoration: const BoxDecoration(
              color: Color(0xFFF9FAFB),
              borderRadius: BorderRadius.only(
                topLeft: Radius.circular(12),
                topRight: Radius.circular(12),
              ),
              border: Border(
                bottom: BorderSide(color: Color(0xFFEDF2F7)),
              ),
            ),
            child: Row(
              children: [
                Icon(
                  icon,
                  size: 22,
                  color: const Color(0xFF4A5568),
                ),
                const SizedBox(width: 12),
                Text(
                  title,
                  style: const TextStyle(
                    fontSize: 16,
                    fontWeight: FontWeight.w600,
                    color: Color(0xFF2D3748),
                  ),
                ),
              ],
            ),
          ),
          Padding(
            padding: padding,
            child: child,
          ),
        ],
      ),
    );
  }
}

class _FormDropdown<T> extends StatelessWidget {
  final String label;
  final double width;
  final T? value;
  final List<T> items;
  final void Function(T?)? onChanged;
  final String Function(T) displayText;
  final String hintText;
  const _FormDropdown({
    Key? key,
    required this.label,
    required this.width,
    required this.value,
    required this.items,
    required this.onChanged,
    required this.displayText,
    this.hintText = 'Select...',
  }) : super(key: key);
  @override
  Widget build(BuildContext context) {
    return SizedBox(
      width: width,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            label,
```

```
            style: const TextStyle(
              fontSize: 11,
              fontWeight: FontWeight.w600,
              color: Color(0xFF4A5568),
            ),
          ),
          const SizedBox(height: 8),
          Container(
            height: 36,
            padding: const EdgeInsets.symmetric(horizontal: 10),
            decoration: BoxDecoration(
              color: Colors.white,
              borderRadius: BorderRadius.circular(6),
              border: Border.all(color: const Color(0xFFCBD5E0)),
              boxShadow: [
                BoxShadow(
                  color: Colors.black.withOpacity(0.02),
                  blurRadius: 4,
                  offset: const Offset(0, 2),
                ),
              ],
            ),
            child: DropdownButtonHideUnderline(
              child: DropdownButton<T>(
                hint: Text(
                  hintText,
                  style: const TextStyle(fontSize: 12, color: Colors.grey),
                ),
                value: value,
                isExpanded: true,
                icon:
                    const Icon(Icons.arrow_drop_down, color: Color(0xFF4A5568)),
                items: items.map((item) {
                  return DropdownMenuItem<T>(
                    value: item,
                    child: Text(
                      displayText(item),
                      style: const TextStyle(fontSize: 12),
                      overflow: TextOverflow.ellipsis,
                    ),
                  );
                }).toList(),
                onChanged: onChanged,
                style: const TextStyle(
                  fontSize: 12,
                  color: Color(0xFF1A202C),
                ),
              ),
            ),
          ),
        ],
      ),
    );
  }
}

class _FormTextField extends StatelessWidget {
  final String label;
  final double width;
  final bool readOnly;
  final String? hintText;
  final TextEditingController? controller;
  const _FormTextField(
    this.label, {
    super.key,
    required this.width,
    this.readOnly = false,
    this.hintText,
    this.controller,
  });
  @override
  Widget build(BuildContext context) {
    return SizedBox(
      width: width,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            label,
            style: const TextStyle(
              fontSize: 11,
              fontWeight: FontWeight.w600,
              color: Color(0xFF4A5568),
            ),
```

```
          ),
          const SizedBox(height: 8),
          Container(
              height: 36,
              decoration: BoxDecoration(
                borderRadius: BorderRadius.circular(6),
                boxShadow: [
                  BoxShadow(
                    color: Colors.black.withOpacity(0.02),
                    blurRadius: 4,
                    offset: const Offset(0, 2),
                  ),
                ],
              ),
              child: TextFormField(
                controller: controller,
                readOnly: readOnly,
                style: const TextStyle(
                  fontSize: 12,
                  color: Colors.black,
                  fontWeight: FontWeight.w500,
                ),
                decoration: InputDecoration(
                  filled: true,
                  fillColor: readOnly ? const Color(0xFFF7F8FA) : Colors.white,
                  contentPadding: const EdgeInsets.symmetric(horizontal: 10),
                  hintText: (hintText?.isNotEmpty ?? false) ? hintText : '-',
                  hintStyle: TextStyle(
                    fontSize: 12,
                    color: readOnly ? const Color(0xFF2D3748) : Colors.grey,
                  ),
                  border: OutlineInputBorder(
                    borderRadius: BorderRadius.circular(6),
                    borderSide: const BorderSide(color: Color(0xFFCBD5E0)),
                  ),
                  enabledBorder: OutlineInputBorder(
                    borderRadius: BorderRadius.circular(6),
                    borderSide: const BorderSide(color: Color(0xFFCBD5E0)),
                  ),
                  focusedBorder: OutlineInputBorder(
                    borderRadius: BorderRadius.circular(6),
                    borderSide: const BorderSide(
                      color: Color(0xFF005FB8),
                      width: 1.5,
                    ),
                  ),
                ),
              )),
        ],
      ),
    );
  }
}

// --- Data Models
class _IrrOption {
  const _IrrOption({required this.irrCode, required this.refNo});
  final String irrCode;
  final String refNo;
  factory _IrrOption.fromJson(Map<String, dynamic> json) {
    final rawIrrCode = '${json['IRR_CODE'] ?? ''}';
    final actualIrrCode =
        rawIrrCode.contains('~') ? rawIrrCode.split('~')[0] : rawIrrCode;
    return _IrrOption(
      irrCode: actualIrrCode,
      refNo: '${json['REF_NO'] ?? json['PLEDGE_NO'] ?? ''}',
    );
  }
  @override
  bool operator ==(Object other) =>
      other is _IrrOption &&
      other.irrCode == irrCode &&
      other.refNo == refNo;
  @override
  int get hashCode => Object.hash(irrCode, refNo);
}

class _AmbienceDetails {
  const _AmbienceDetails({
    this.irrcode,
    this.irregularityDate,
    this.status,
    this.rectificationDate,
    this.category,
```

```
  });

  final String? irrcode;
  final String? irregularityDate;
  final String? status;
  final String? rectificationDate;
  final String? category;

  factory _AmbienceDetails.fromJson(Map<String, dynamic> json) => _AmbienceDetails(
        irrcode: json['IRR_CODE']?.toString(),
        irregularityDate: json['IRR_RPT_DATE']?.toString(),
        status: json['STATUS']?.toString(),
        rectificationDate: json['CLOSE_DT']?.toString(),
        category: json['CATEGORY']?.toString(),
      );
}
```